Instituto Tecnológico de Costa Rica
Ingeniería en Computación
Compiladores e Intérpretes
Semestre I, 2026
Profesor: Allan Rodríguez Dávila

Proyecto #1
Análisis Léxico y Sintáctico

Introducción

Un grupo de desarrolladores desea crear un nuevo lenguaje imperativo, ligero, que le permita realizar operaciones básicas para la configuración de chips, ya que esta es una industria que sigue creciendo constantemente, y cada vez estos chips necesitan ser configurados por lenguajes más ligeros y potentes. Es por esto por lo que este grupo de desarrolladores requiere desarrollar su propio lenguaje para el desarrollo de sistemas empotrados.

Proyecto a desarrollar

Este proyecto comprende la fase de análisis léxico y análisis sintáctico para la gramática descrita en el Anexo I (descrita también en Tarea I). Se debe desarrollar un scanner utilizando la herramienta JFlex y se debe desarrollar un parser utilizando la herramienta Cup.

Un programa escrito para este lenguaje está compuesto por una secuencia de declaraciones de procedimientos, que contienen diferentes expresiones y asignaciones de expresiones; todo programa debe contener un único método main.

Para comprobar el desarrollo de ambas fases el programa a presentar deberá tomar un archivo fuente y realizar lo siguiente:

a) El sistema debe leer un archivo fuente.
b) Se debe escribir en un archivo todos los tokens encontrados, identificador asociado con el lexema.

c) Por cada token deberán indicar en cuál tabla de símbolos va y cual información se almacenará.
d) Indicar si el archivo fuente puede o no ser generado por la gramática.
e) Reportar y manejar los errores léxicos y sintácticos encontrados. Debe utilizar la técnica de Recuperación en Modo Pánico (error en línea y continúa con la siguiente). **En el reporte de errores es fundamental indicar la línea en que ocurre.**

Gramática
La gramática BNF que reconocerá el lenguaje será la descrita en el Anexo I que fue desarrollada en la tarea I. La gramática desarrollada en la tarea puede ser sujeta a cambios según se avance en la implementación de las dos primeras fases del compilador. La gramática puede tener ideas de gramáticas de referencia, pero en su estructura debe ser original.

Scanner
El desarrollo del Analizador Léxico se debe realizar utilizando la herramienta JFLex. Incluye el reconocimiento de los tokens y recuperación de errores sobre los archivos fuentes analizados. Debe reportar los errores y seguir procesando el código fuente. Tiene un valor de 40 puntos.

Parser
El desarrollo del Analizador Sintáctico se debe realizar utilizando la herramienta Cup. Incluye la comprobación sintáctica y recuperación de errores. Debe reportar los errores y seguir procesando el código fuente. Tiene un valor de 40 puntos

Puntos Extra

Se darán 2.5 puntos extra al implementar técnicas de recuperación de errores del tipo Recuperación a nivel de frase, Producción de Errores o Corrección Global.

Se darán **2.5 puntos adicionales** al entregar a más tardar el miércoles 15 de abril a las 11:55:55 PM el Documento de Requerimientos Detallado, ver plantilla suministrada en el Tec Digital. Debe subirse en la documentación llamada “Proyecto I (archivos adicionales)” debajo de la carpeta de “Proyectos”.
Requerimientos.PrimerNombreMiembro1.PrimerNombreMiembro2.zip

Aspectos técnicos

El proyecto deberá correr en java y utilizar las herramientas JFlex y Cup. En caso de requerir librerías adicionales para compilar y ejecutar el programa, deberán especificarlo en la documentación, ya que de lo contrario se descontarán puntos en la evaluación. Se debe incluir los archivos .flex y .cup, y el proyecto en Java que utilice los archivos generados por JFlex y Cup.

Deberán utilizar el sistema de control de versiones GitHub, el repositorio deberá ser público o incluir al profesor en el control de acceso de este.

Se valorará el aporte generado por cada estudiante, considerando, entre otras cosas, los commit generados por cada uno. Por lo que el puntaje obtenido por cada uno de los estudiantes puede ser diferenciado.

<u>Cualquier producción adicional que se asocie a otros lenguajes o gramáticas externas acarreará penalización.</u>

<u>El uso de producciones gramaticales generadas por inteligencia artificial acarreará penalización.</u>

Documentación

La documentación es un aspecto de gran importancia en el desarrollo de programas, especialmente en tareas relacionadas con el mantenimiento de estos.

Para la documentación interna, deberán incluir comentarios descriptivos para cada parte, con sus entradas, salidas, restricciones y **objetivo**.

La documentación externa deberá incluir:

1. Portada.
2. Manual de usuario: **instrucciones de compilación, ejecución y uso bien detalladas**.
3. Pruebas de funcionalidad: incluir *screenshots*.
4. Descripción del problema.
5. Diseño del programa: decisiones de diseño, algoritmos usados.
6. Librerías usadas: creación de archivos, etc.

7. Análisis de resultados: objetivos alcanzados, objetivos no alcanzados, y razones por las cuales no se alcanzaron los objetivos (en caso de haberlos).
8. Justificación de toma de decisiones: por qué se generaron las producciones en expresiones y la precedencia.
9. Bitácora (autogenerada en git, commit por usuario incluyendo comentario).

Evaluación

La evaluación se va a centrar en dos elementos: programación y documentación.

El proyecto programado tiene un valor de **12.5%** de la nota final, en el rubro de Proyectos.

Desglose de la evaluación del proyecto programado:

1. Documentación interna 2 ptos.
2. Documentación externa 6 ptos.
3. Funcionalidad 82 ptos (ver detalle en Proyecto a Desarrollar)
4. Revisión del proyecto (estado del sistema y correcta gestión del tiempo) 5 ptos.
5. Hora de Entrega 5 ptos.

Forma de trabajo

El trabajo se debe realizar en parejas.

Aspectos administrativos

Debe crear un archivo **.zip** ("PP1_Integrante1_Integrante2.zip") que contenga únicamente un archivo **info.txt** y 2 carpetas llamadas **documentacion** y **programa**, en la primera deberá incluir el documento de *word* o pdf solicitado y en la segunda los archivos y/o carpetas necesarias para la implementación de este proyecto programado. El archivo **info.txt** debe contener la siguiente información (cualidades):

a. Nombre del curso

b. Número de semestre y año lectivo
c. Nombre de los Estudiantes
d. Número de carnet de los estudiantes
e. Número de proyecto programado
f. Fecha de entrega
g. Estatus de la entrega (debe ser **CONGRUENTE** con la solución entregada): [Deplorable|Regular|Buena|MuyBuena|Excelente|Superior]

Entrega

Deberá subir el archivo antes mencionado al TEC Digital en el curso de COMPILADORES E INTERPRETES GR 60, en la asignación llamada "P1" debajo del rubro de "Proyectos". En la evaluación del Proyecto el rubro de "Hora de Entrega" valdrá por 5 puntos de la nota total del proyecto, según la siguiente escala:

a. Si se entrega antes de las 11:55:55 **PM** del lunes 27 de abril de 2026, 5 puntos.
b. Si se entrega antes de las 11:55:55 **AM** del martes 28 de abril de 2026, 2.5 puntos.
c. Si se entrega antes de las 11:55:55 **PM** del martes 28 de abril de 2026, 0 puntos. Después de este punto, **NO SE ACEPTARÁN** más trabajos.

Los archivos .flex y .cup pueden ser revisados en el sistema de Control de Plagio del TEC Digital.

Todo el contenido de cada proyecto debe ser 100% original y en caso de conducta fraudulenta se procederá según lo indicado en el artículo 75 del RREA. Todos los miembros del grupo deberán participar en el desarrollo del proyecto y en la revisión, ya que de lo contrario se les penalizará con puntos.

ANEXO I

GRAMÁTICA

La Gramática BNF (considerando únicamente **aspectos léxicos y sintácticos**) para un lenguaje imperativo con las siguientes características:

a. Permitir la creación de funciones secuenciales, y dentro de ellas, estructuras de control, bloques de código (|: y :|) y sentencias de código.
b. Manejar los tipos de variables enteras, flotantes, booleanas, caracteres, cadenas de caracteres (string), arreglo estático bidimensional, entero notación exponencial con e –con exponente positivo– (ejemplo 100e10), fraccionario simple –variación de flotante– (ejemplo 5/6).
c. Se permite crear arreglos de tipo flotantes y entero. Además, se permite obtener y modificar sus elementos, y ser utilizados en expresiones. Se permite creación con asignación (|: y :|). Los índices se indican con << y >>.
d. Permitir sentencias para creación de variables locales, creación y asignación de expresiones y asignación de expresiones a variables, y algunos casos, sólo expresiones sin asignación. Observaciones:
    i. En el caso de creaciones utilizar el separador ~ entre tipo e identificador
    ii. El símbolo de asignación será <-
e. Las expresiones permiten combinar literales, variables, arreglos y/o funciones, de los tipos reconocidos en la gramática.
f. Debe permitir operadores y operandos, respetando precedencia (usual matemática) y permitiendo el uso de paréntesis (<| y |>).
g. Permitir expresiones aritméticas binarias de suma (+), resta (-), división (/) –entera o decimal según el tipo--, multiplicación (*), módulo (%) y potencia (^). Para enteros o flotantes según aplique.
h. Permitir expresiones aritméticas unarias de negativo (-), ++ y -- antes del operando; el negativo se puede aplicar a literales enteros y flotantes, el ++ y -- se aplica a variables enteres y flotantes.
i. Permitir expresiones relacionales (sobre enteros y flotantes) igual, diferente, menor, menor o igual, mayor, mayor o igual, de la siguiente forma:
    i. equal<|op1,op2|>
    ii. n_equal<|op1,op2|>
    iii. less_t<|op1,op2|>
    iv. less_te<|op1,op2|>
    v. greather_t<|op1,op2|>
    vi. greather_te<|op1,op2|>

    Estos operadores deben estar relacionados a nivel de gramática con los operadores aritméticos y lógicos.

    Los operadores igual y diferente permiten adicionalmente tipo booleano.
j. Permitir expresiones lógicas de conjunción (@), disyunción (#) y negación (ésta debe ser de tipo caracter ($)).
k. Debe permitir sentencias de código para las diferentes expresiones mencionadas anteriormente y su combinación, el delimitador de final de expresión será el

carácter admiración (!). Además, dichas expresiones pueden usarse en las condicionales y bloques de las siguientes estructuras de control.

l. Debe permitir el uso de tipos y la combinación de expresiones aritméticas (binarias y unarias), relacionales y lógicas, según las reglas gramaticales, aritméticas, relacionales y lógicas del Paradigma Imperativo, por ejemplo, tomando como referencia el lenguaje C.
   Las producciones deben combinar los tipos de expresiones.
   Cualquier duda con respecto al comportamiento de la funcionalidad debe validarlo con el profesor.
m. La gramática genera un lenguaje con tipado explícito y fuerte.
n. Debe permitir las estructuras de control if-else, do-while, switch.
   a. Además, permitir **return** (utilizar separador con la expresión) y **break**.
      Las expresiones de las condiciones deberán ser valores booleanos combinando expresiones aritméticas, lógicas y relacionales.
o. Debe permitir las funciones de leer (enteros y flotantes) y escribir en la salida estándar (cadena carácter, enteros, boolean y flotantes), se pueden escribir literales o variables. Utilizar palabras reservadas **cin** y **cout**.
p. Debe permitir la creación y utilización de funciones, estos deben retornar valores (entero, flotantes, char y booleanos) y recibir parámetros (con tipo). Utilizar separador ~, coma, <| y |>.
q. Debe definir un único procedimiento inicial main (llamado `__main__`), por medio de la cual se inicia la ejecución de los programas, este es de tipo `empty` y no recibe parámetros.
r. Además, debe permitir comentarios de una línea (¡¡) o múltiples líneas ({- y -}).
s. En los casos que exista una secuencia entre dos palabras reservadas o una palabra reservada e identificador, en el medio debe colocar el separador ~

Otras consideraciones

1. Los siguientes tipos de gramáticas para expresiones están prohibidos:

```
exp ::= or_exp
or_exp ::= or_exp "||" and_exp
                | and_exp
and_exp ::= and_exp "&&" not_exp
                | not_exp
not_exp ::= "!" not_exp
                | rel_exp
```

o

```
expr ::= expr SUMA expr
     | expr RESTA expr
     | expr MULTIPLICACION expr
     | expr DIVISION expr
     | expr DIVISION_ENTERA expr
     | expr MODULO expr
     | expr POTENCIA expr
```

| expr MAYOR expr
| expr MENOR expr
| expr IGUAL expr
| expr DIFERENTE expr
| expr MAYOR_IGUAL expr
| expr MENOR_IGUAL expr
| expr AND expr
| expr OR expr

t.